

Aim:

A government is planning a nationwide high-speed fiber internet network connecting multiple cities. Each possible cable connection between cities has a different installation cost. The government wants to connect all cities while minimizing the total construction cost. Your task is to determine the minimum total cost required to connect all cities using Kruskal's Minimum Spanning Tree Algorithm.

Input Format:

1. The First line contains two space-separated integers V and E indicating number of cities and number of possible connections.

2. Next E lines each contain three space-separated integers u , v , w .
where,

- $u \rightarrow$ Start city
- $v \rightarrow$ Destination city
- $w \rightarrow$ Cost of the connection between the two cities

Output Format:

- A single integer representing the minimum total cost to connect all cities using Kruskal's Minimum Spanning Tree (MST) algorithm.

Note: Partial code is provided. Complete the missing logic in `find()`, `unionSet()`, `compare()`, and the **MST** processing logic.

Source Code:

`fiberCostKrushkal.c`

```
#include <stdio.h>
#include <stdlib.h>

struct Edge {
    int u,v,w;
    // Define the data members required for an edge
};

int parent[100005];

int find(int x) {
    if(parent[x]==x)
        return x;
    return parent[x]=find(parent[x]);
    // Return the representative element of the set
}

void unionSet(int a, int b) {
    parent[find(a)]=find(b);
    // Combine two sets into one
}

int compare( const void *a,const void *b ) {
```

```

    struct Edge *e1=(struct Edge *)a;
    struct Edge *e2=(struct Edge *)b;
    return e1->w-e2->w;

    // Define the sorting criterion

}

int main() {
int V, E;
scanf("%d %d", &V, &E);
struct Edge edges[E];
for(int i = 0; i < E; i++)
scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
for(int i = 0; i < V; i++)
parent[i] = i;
qsort(edges, E, sizeof(struct Edge), compare);
int mstCost = 0;

    // Process the input data as required
    for(int i=0;i<E;i++){
        int u=edges[i].u;
        int v=edges[i].v;

        if(find(u)!= find(v)){
            mstCost += edges[i].w;
            unionSet(u, v);
        }
    }
    // Update the result based on the algorithm logic

printf("%d\n", mstCost);
return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
19

Test Case - 2
User Output
3 3

0 1 1
1 2 2
0 2 3
3